

Transformation et manipulation de données



Dr Khadidja Henni

Séance 11: Traitement des images

Plan de cours

- Analyse et apprentissage des données visuelles
 - Notions sur la formation d'images
 - Amélioration de la qualité des images
 - Extraction des attributs visuels dans les images
 - Apprentissage des données visuelles
 - Apprentissage profond pour la reconnaissance visuelle
 - Quelques applications
-
- Laboratoire formatif

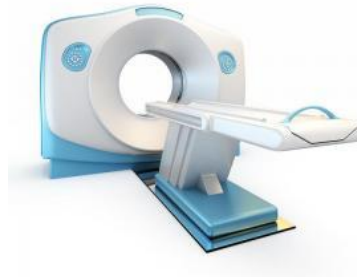
Analyse et apprentissage de données visuelles

Introduction

- Avec l'accessibilité des **caméras numériques** à bas prix, le nombre d'images et de vidéos générées quotidiennement augmente avec une vitesse fulgurante.
- On estime que 73% du contenu du Web est peuplé par des images et des vidéos.
- **Les caméras de surveillance** génèrent des milliers d'heures d'enregistrements vidéo.
- Les **maisons, voitures, robots intelligents** se baseront de plus en plus sur **les technologies visuelles**.

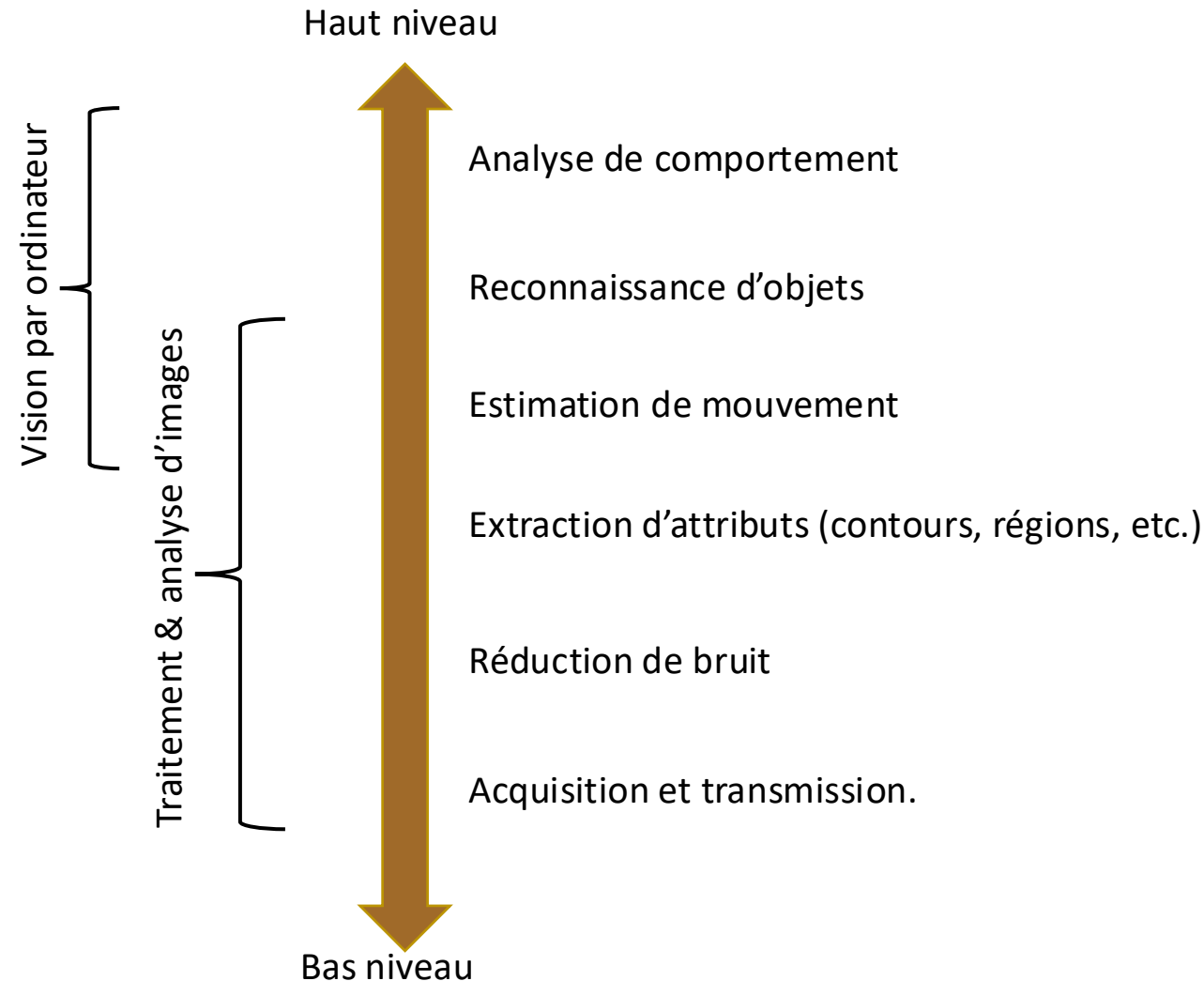
Apprentissage visuel

- Comment comprendre et interpréter le contenu des images comme les humains?



- Réponse: **intelligence artificielle**?

Traitement des données visuelles

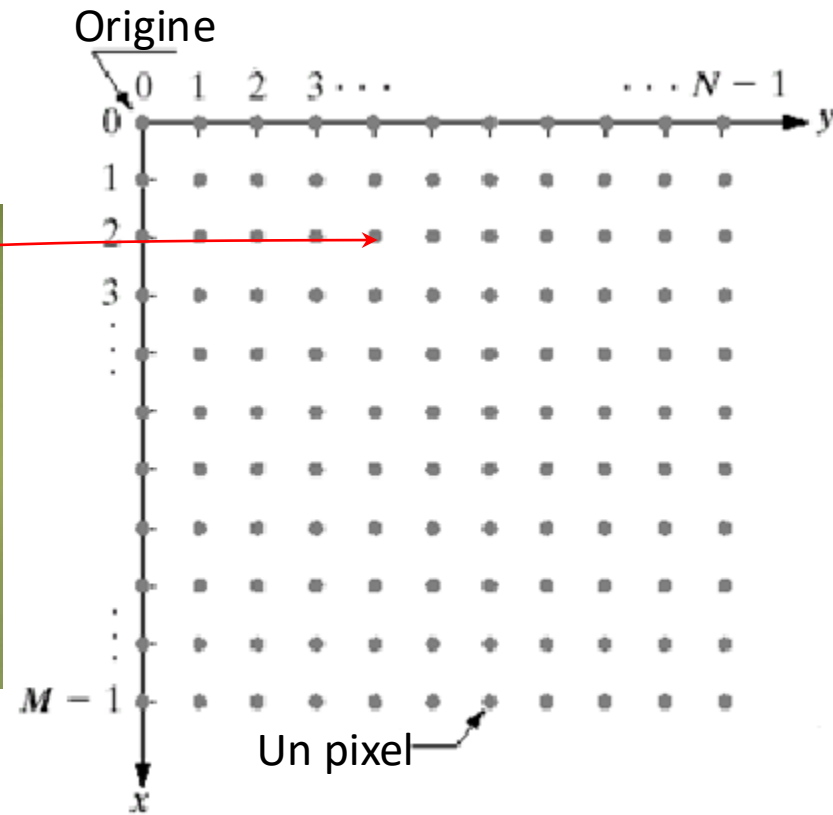


Où se situe l'analyse d'images?

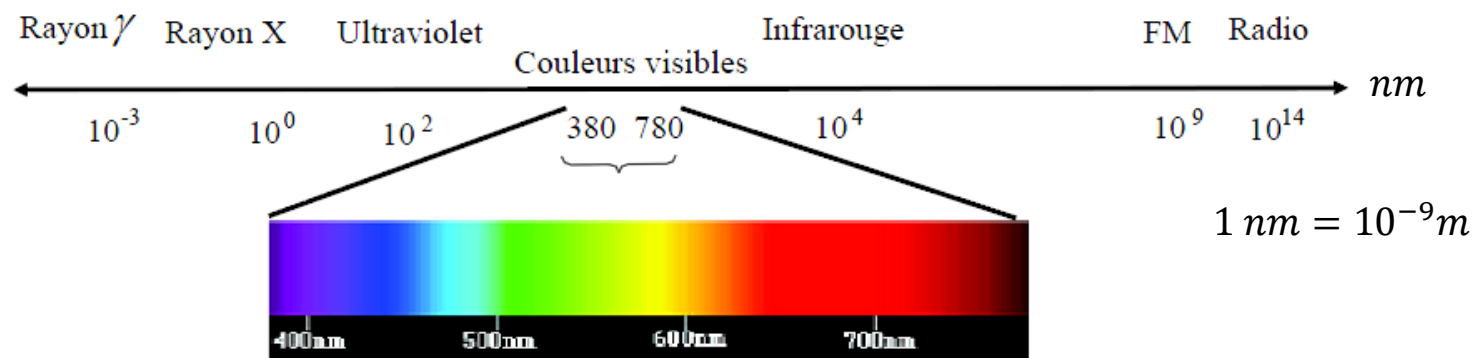
Notions sur la formation d'images

Formation d'images

- Une image I est un tableau (matrice) de $M \times N$ pixels .



Formation d'images

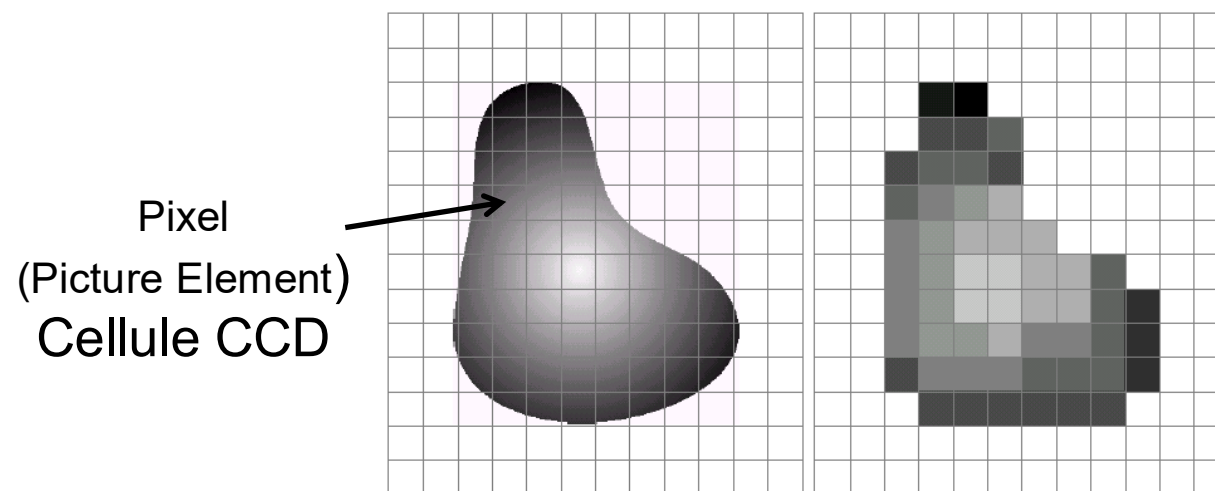


La petite portion qu'on peut voir avec nos yeux →

les **couleurs visibles** (≈ de **380 nm** pour le violet à **780 nm** pour le rouge).

Spectre des fréquences de couleurs visibles (3 bandes principales: **R**, **G**, **B**)

- Une image numérique est constituée d'une **grille de petits carrés** appelés **pixels** (*Picture Elements*).
- Chaque pixel contient une **valeur d'intensité lumineuse** (en niveaux de gris ici, mais ça peut être en couleur R, G, B).
- Plus il y a de pixels (résolution élevée), plus l'image est **fine et précise**. Moins il y en a, plus l'image est **grossière**.



Projection de chaque bande sur le plan de l'image

Quantification et codage de la couleur

- **Capture de l'énergie lumineuse:** Chaque point d'un objet 3D réfléchit ou émet une **énergie lumineuse** appartenant au **spectre électromagnétique** (dans la zone visible: 380–780 nm).
- **Projection sur le capteur:**
 - Cette énergie arrive sur le capteur (CCD ou CMOS).
 - Chaque cellule du capteur correspond à un **pixel** et génère un **courant électrique proportionnel à l'intensité reçue**.
- **Quantification**
 - Le signal analogique est converti en valeurs numériques.
 - Exemple courant : **256 niveaux par bande** (de 0 à 255).
- **Codage des pixels**
 - Chaque pixel (x, y) possède donc une valeur **dans chaque canal R, G et B**.
 - Avec 3 canaux × 8 bits = **24 bits par pixel** → possibilité de représenter ≈ **16,7 millions de couleurs**.

Exemple

- Noir = (0, 0, 0).
- Blanc = (255, 255, 255).
- Rouge pur = (255, 0, 0).
- Vert pur = (0, 255, 0).
- Bleu pur = (0, 0, 255).

Quantification et codage de la couleur

- Un pixel d'une image couleur est stocké sur 24 bits.



Quantification et codage de la couleur

- Il existe aussi des images à niveau de gris où la couleur n'est pas codée. Un pixel est codé sur 8 bits seulement.

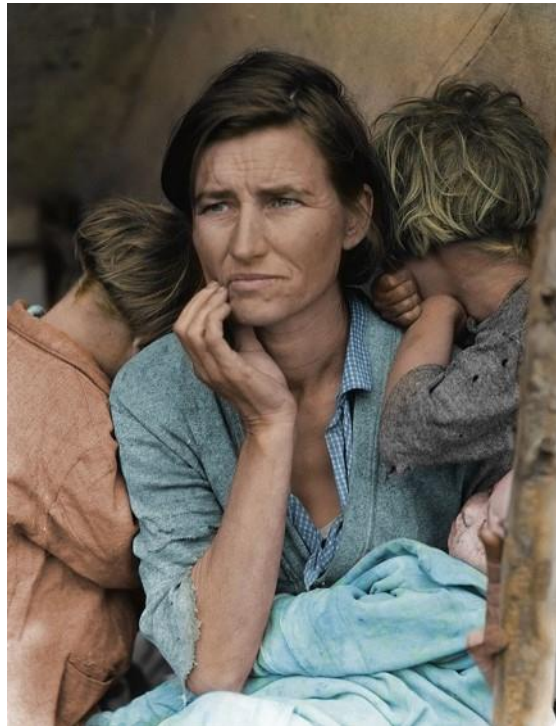


image couleur

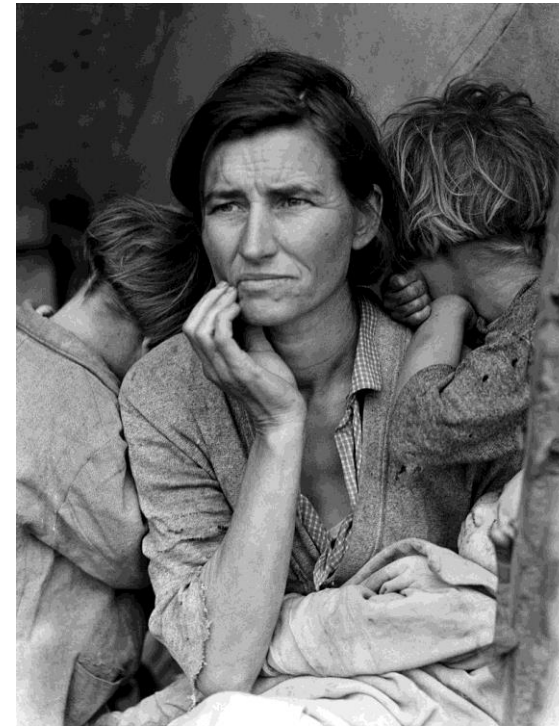


Image à niveau de gris

Amélioration de la qualité des images

Introduction

- Avant tout traitement d'images, on peut corriger **les erreurs introduites par le processus d'acquisition** des images (ex. bruit, flou, luminosité, distorsions géométriques, etc.).
- Plusieurs opérations peuvent être envisagées sur une image, dépendamment de **l'amélioration** ciblée:
 - ➡ Correction de la luminosité.
 - ➡ Réduction de bruit.
 - ➡ Amélioration du contraste.
 - ➡ Correction géométrique, etc.

Correction de la luminosité

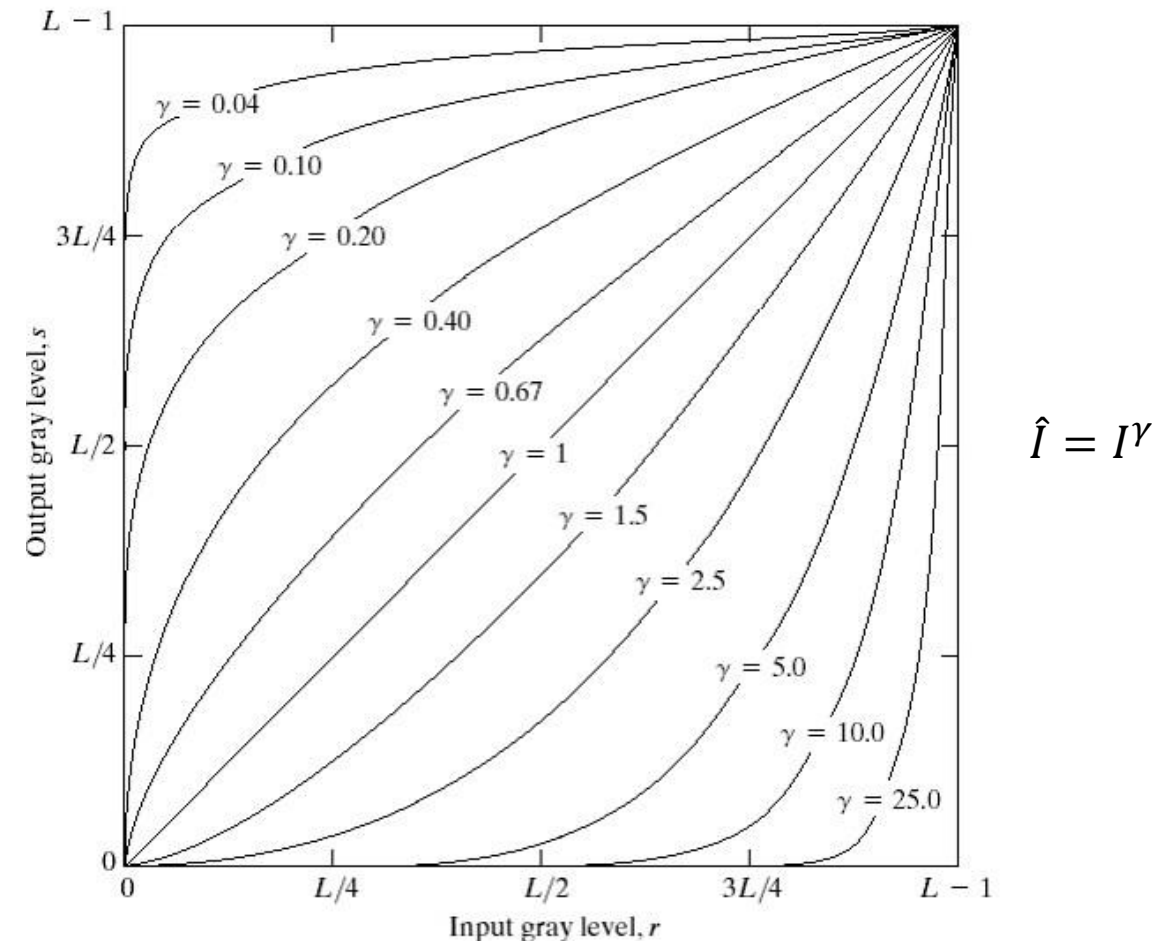
Pourquoi ?

Quand une image est trop sombre (sous-exposée) ou trop claire (surexposée), les détails deviennent difficiles à distinguer.

Exemple : une photo prise de nuit sans flash ou une photo au soleil avec trop d'éclat.

Principe

- Chaque **pixel** a une intensité comprise entre **0 (noir)** et **255 (blanc)**.
- Corriger la luminosité revient à **ajuster ces valeurs** pour éclaircir ou assombrir l'image.



Graphes de Corrections Gamma

Correction de la luminosité

Méthodes principales

- **Translation linéaire** : on ajoute une valeur constante $\beta \rightarrow \text{nouveau_pixel} = \text{ancien_pixel} + \beta$.
 - $\beta > 0 \rightarrow$ éclaircit, $\beta < 0 \rightarrow$ assombrit.
- **Correction gamma** : on applique une fonction non linéaire

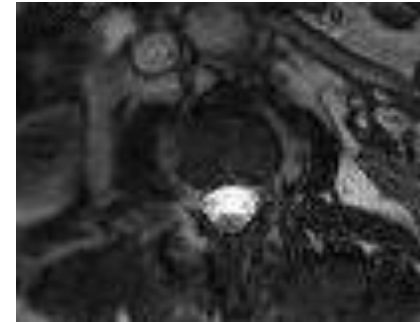
$$I_{\text{corrigé}} = 255 \times \left(\frac{I_{\text{original}}}{255} \right)^{\frac{1}{\gamma}}$$

- $\gamma > 1 \rightarrow$ éclaircit les zones sombres sans saturer les zones claires.
- $\gamma < 1 \rightarrow$ assombrit.

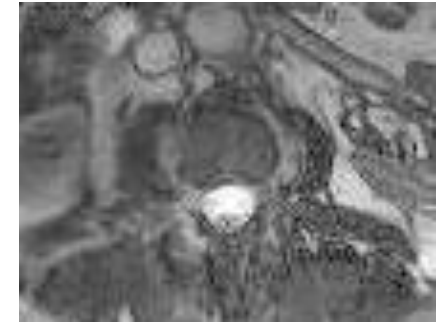
Exemple visuel

- Image trop sombre $\rightarrow$ on augmente la luminosité.
- Image trop claire $\rightarrow$ on réduit la luminosité.

Image



Correction de la luminosité



Correction de la luminosité

```
import cv2
import numpy as np
import os
from google.colab.patches import cv2_imshow
```

```
def gamma_correction(img_bgr, gamma=1.3):
    inv_gamma = 1.0 / gamma
    table = np.array([(i/255.0)**inv_gamma * 255 for i in range(256)]).astype("uint8")
    return cv2.LUT(img_bgr, table)
```

Gamma correction - Avant / Après



Réduction de bruit

- Un **bruit** est toute information **non-pertinente et nuisible** ajoutée aux données utiles.

Image sans bruit



Image avec bruit



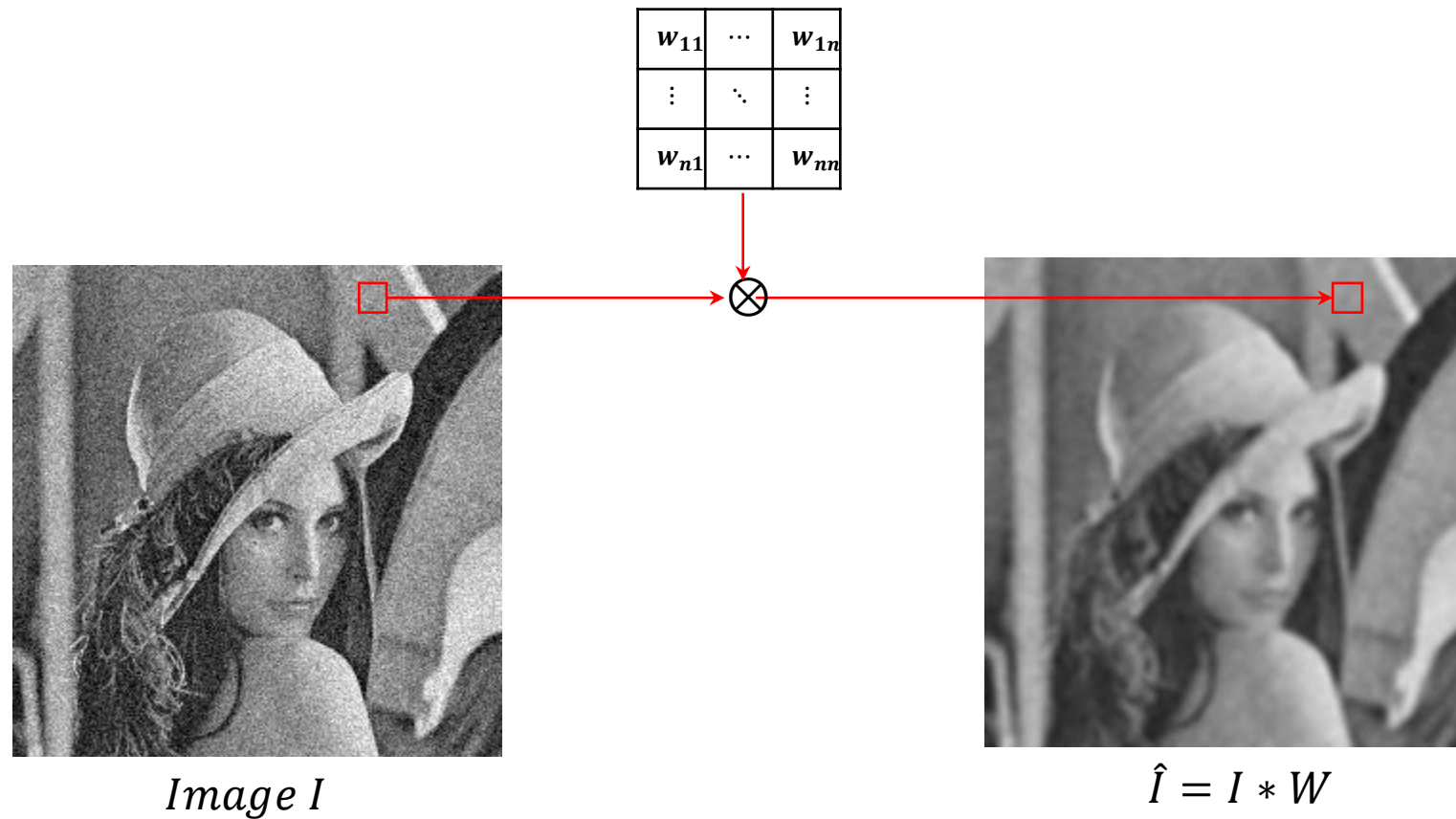
Réduction de bruit

- Pour réduire le bruit dans une image I , on peut utiliser des techniques de filtrage d'image.
- Un filtre est une image avec des poids.

$$W = \begin{pmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{n1} & \cdots & w_{nn} \end{pmatrix}$$

- Le filtrage d'une image I consiste à remplacer la valeur de chaque pixel par la somme pondérée de son voisinage.

Réduction de bruit



Réduction de bruit

```
def denoise_nlm_color(img_bgr, h=10, template=7, search=21):  
    return cv2.fastNlMeansDenoisingColored(img_bgr, None, h, h, template, search)
```

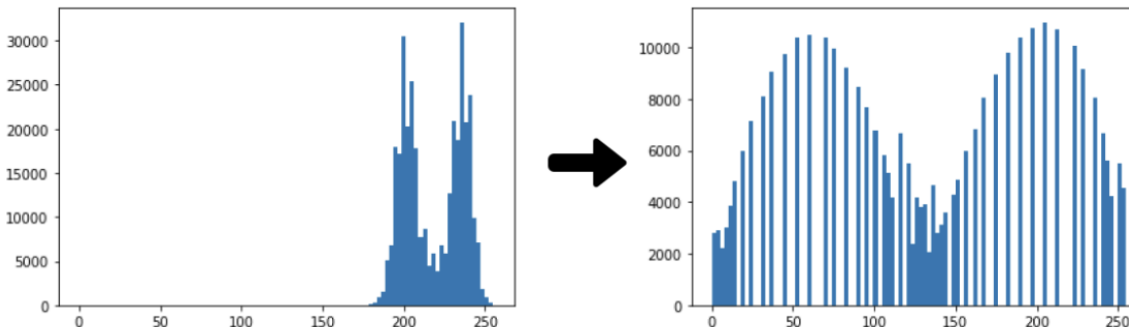
Denoise NLM - Avant / Après



Amélioration du contraste

Est une opération qui consiste à **rehausser les petits détails** de l'image pour améliorer sa qualité visuelle.

1. **Découpage local**: L'image est découpée en petites régions (appelées *tiles*, ex. 8×8).
2. **Égalisation d'histogramme par région** Dans chaque tile, on applique une égalisation d'histogramme pour améliorer le contraste local.
3. **Limitation du contraste** (*clipLimit*) On limite le renforcement pour éviter que le bruit ou les artefacts soient exagérés.
4. **Interpolation** Les tiles sont recollés en douceur par interpolation pour éviter les frontières visibles.



Avant



Après



Amélioration du contraste

3) Contraste CLAHE

```
def equalize_clahe(img_bgr, clipLimit=2.0, tileGridSize=(8,8)):  
    lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)  
    l, a, b = cv2.split(lab)  
    clahe = cv2.createCLAHE(clipLimit=clipLimit, tileGridSize=tileGridSize)  
    l2 = clahe.apply(l)  
    out = cv2.merge([l2, a, b])  
    return cv2.cvtColor(out, cv2.COLOR_LAB2BGR)
```

```
clahe_img = equalize_clahe(denoised)
```

CLAHE (Contraste local) - Avant / Après



Correction géométrique

- **Transformation de similarité:** Inclut **rotation, translation, mise à l'échelle et symétrie**. Elle conserve la forme et les proportions (les angles et rapports de distance restent les mêmes).
- **Transformation affine:** Transformation linéaire plus générale (6 paramètres).
 - Peut inclure **cisaillement (shear)** en plus de la rotation/translation/échelle.
 - Conserve le parallélisme mais pas forcément les angles ni les longueurs.
- **Transformation projective (homographie):** Utilisée pour corriger la **perspective**. Par exemple : redresser une photo de document prise en biais → les lignes parallèles semblent converger, l'homographie corrige cela.
- ...



Avant



Après

Correction géométrique

```
def unsharp_mask(img_bgr, sigma=1.2, amount=1.0):  
    blur = cv2.GaussianBlur(img_bgr, (0,0), sigma)  
    return cv2.addWeighted(img_bgr, 1+amount, blur, -amount, 0)
```

Unsharp Mask (Netteté) – Avant / Après



Extraction des attributs visuels dans les images

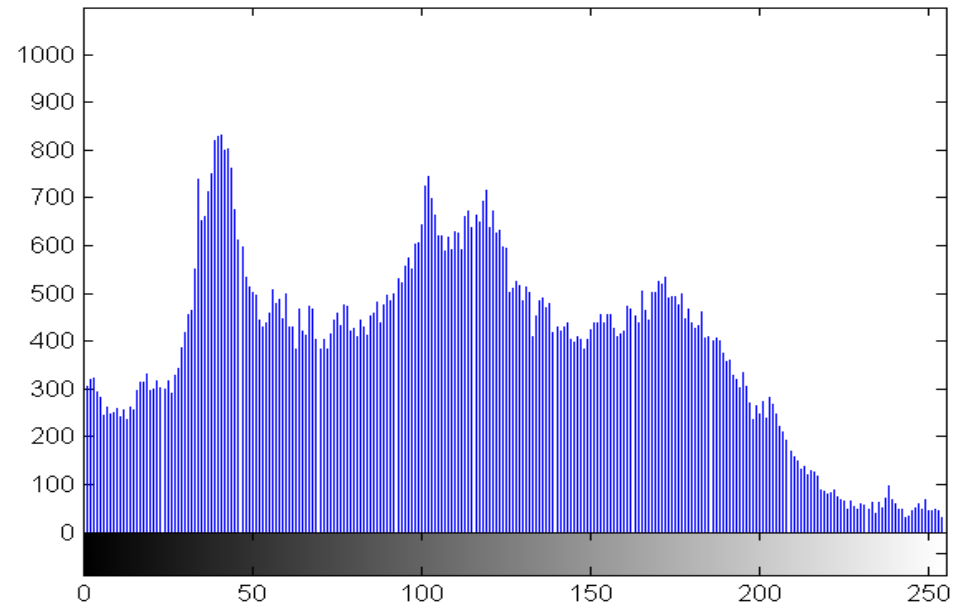
Histogramme d'une image à niveau de gris

L'histogramme donnera un compte du nombre de pixels dans l'image ayant chaque niveau de gris de 0 à 255.

Image



Histogramme



Histogramme d'une image couleur

- **Histogramme d'image couleur**

Pour une image couleur, le nombre de valeurs de couleur possible est $256 \times 256 \times 256 = 2^{24}$ (histogramme trop lourd à calculer!)

- **Solution : la quantification**

On peut réduire le temps de calcul en choisissant une **quantification** pour chaque canal de couleur avec un facteur $k \in \{2, 4, 8, 16\}$.

- On réduit le nombre de niveaux par canal en regroupant les intensités.
- Exemple : Si on choisit $k = 4$, on n'a que $2^4 = 16$ niveaux possibles par canal.
 - Donc au lieu de 256 valeurs, on regroupe les pixels en 16 intervalles.

- Avec $k = 4$: $16 \times 16 \times 16 = 4096$ couleurs représentées dans l'histogramme

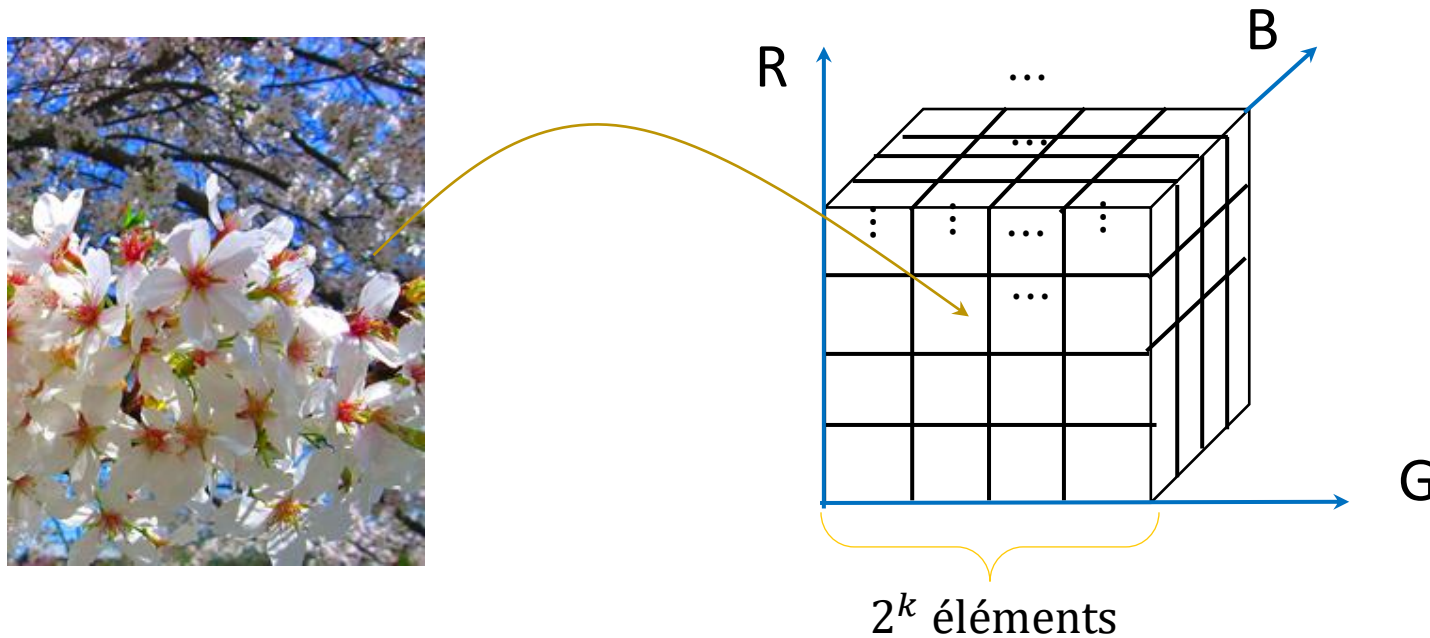
Les valeurs de chaque bande de couleur dans l'ensemble $\{0, \dots, 255\}$ seront qualifiées à des valeurs dans l'ensemble $Q = \{0, \dots, 2^k - 1\}$, avec $k \in \{2, 4, 8, 16\}$.

- **Avantages:**

- Calcul beaucoup plus rapide.
- L'histogramme devient lisible et exploitable pour des tâches comme : Indexation d'images, Recherche par contenu, Comparaison d'images (distances entre histogrammes).

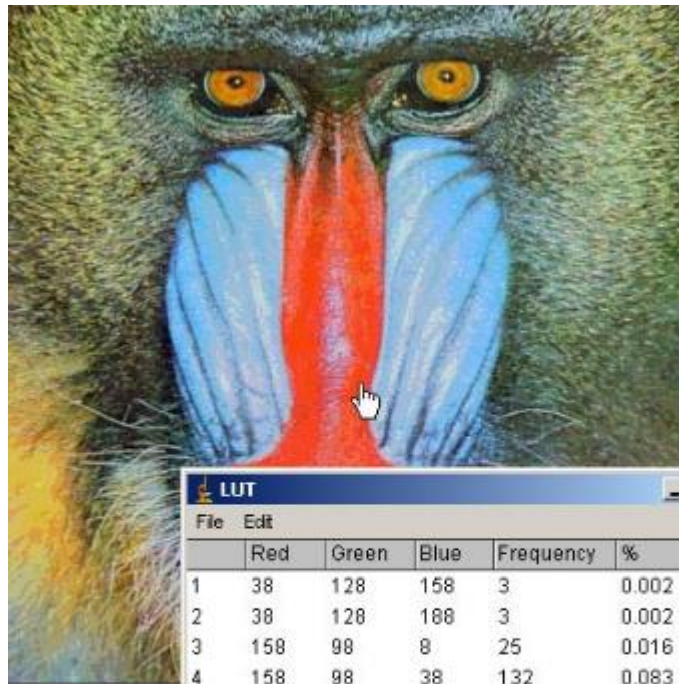
Histogramme d'une image couleur

On construit un cube de fréquences pour chaque couleur dans l'ensemble $Q \times Q \times Q$.



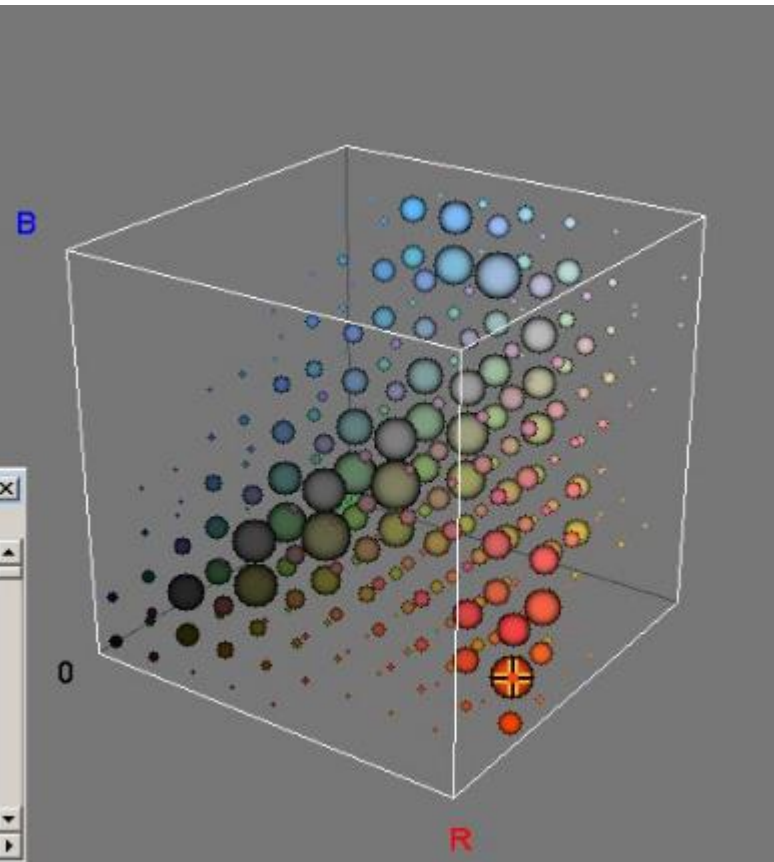
Histogramme d'une image couleur

Image



LUT					
File Edit					
	Red	Green	Blue	Frequency	%
1	38	128	158	3	0.002
2	38	128	188	3	0.002
3	158	98	8	25	0.016
4	158	98	38	132	0.083
5	158	98	68	160	0.100
6	158	98	98	278	0.174
7	158	98	128	54	0.034
8	8	38	8	56	0.035
9	8	28	28	162	0.102

Histogramme



Extraction de contours

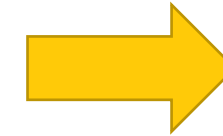
Définition

- L'**extraction de contours** consiste à détecter les **frontières d'objets** dans une image.
- Un contour est une **courbe reliant des points de même intensité**, représentant la limite d'une forme.

Utilité

- Segmenter les objets dans une scène.
- Reconnaissance de formes (pièces mécaniques, visages, etc.).
- Comptage d'objets ou suivi en vision artificielle.

$I(x,y)$



$c(x,y)$



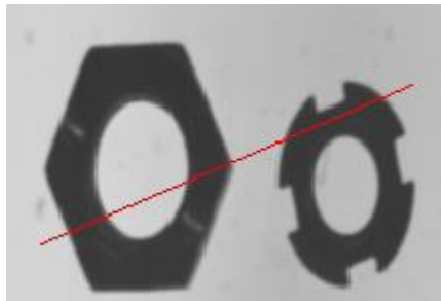
$I(x,y)$: une image d'entrée

$c(x,y)$: une image binaire, $c(i, j) \in \{\text{contour}, \text{non-contour}\}$

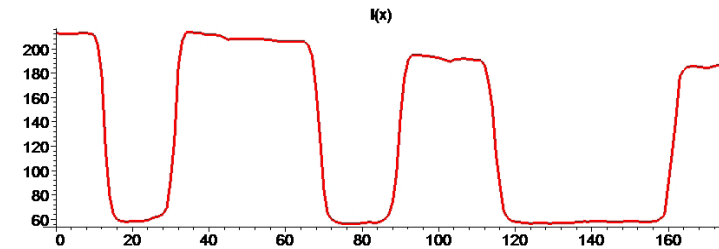
On appelle parfois **edge map** l'image $c(x,y)$ étiquettes

Extraction de contours

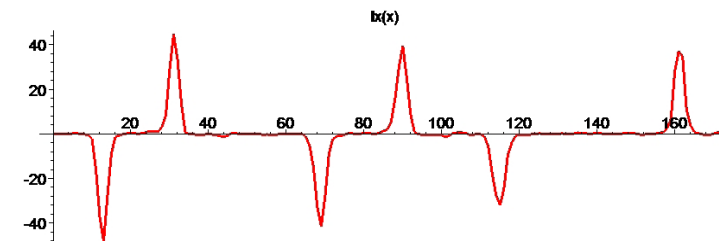
- Un contour correspond à une discontinuité d'intensité.
- Un contour correspond donc à un pique au niveau de la dérivée d'ordre 1.



Une coupe des niveaux gris dans une image à deux dimensions



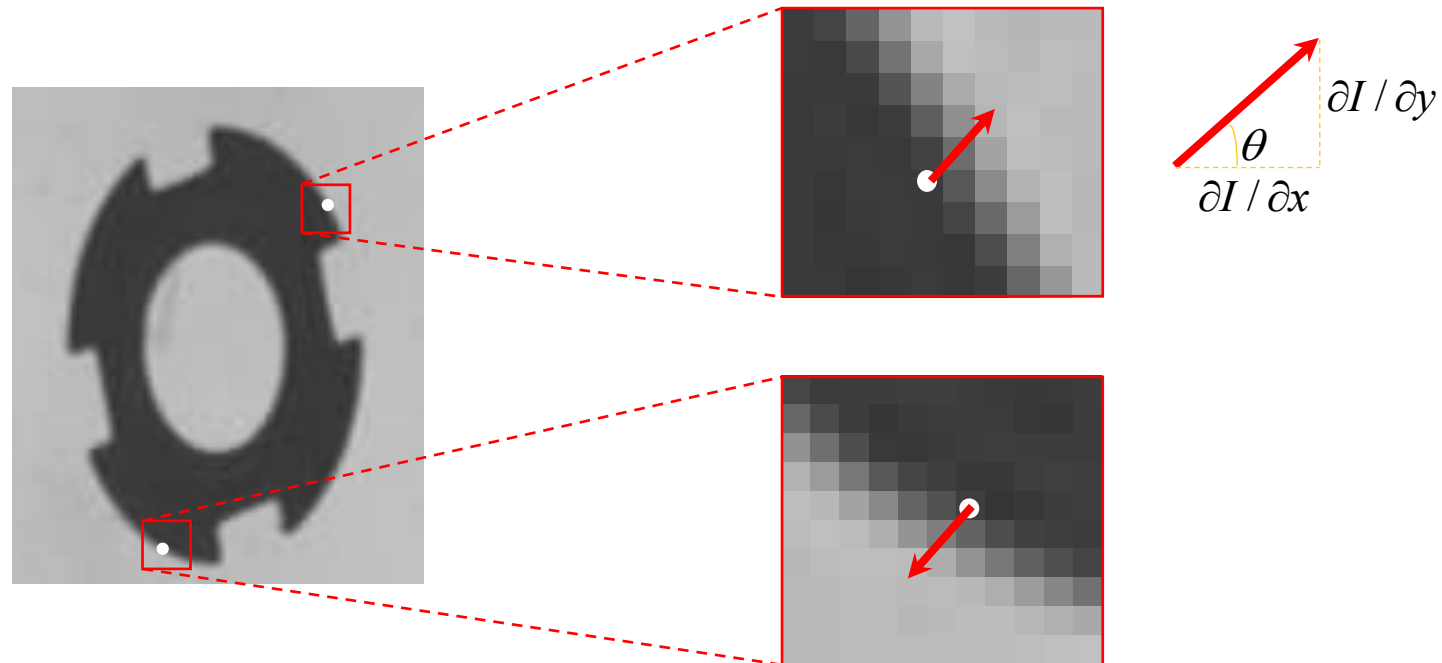
Niveaux de gris le long de la ligne rouge



Dérivée d'ordre 1 des niveaux de gris. Les contours sont les maxima positifs et les minima négatifs

Extraction de contours

Le gradient de l'image pointe toujours en **direction du changement maximal**.

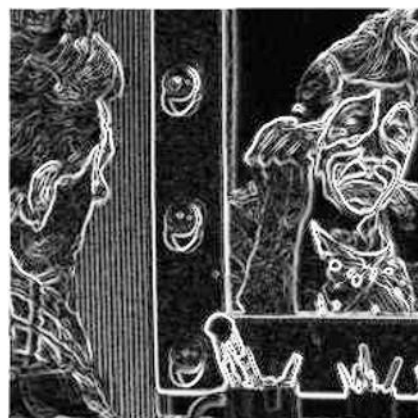


Plus l'**amplitude** du gradient est **forte**, plus la discontinuité est **brusque**.

Extraction de contours



Image



Module gradient



Après seuillage



Post - traitement

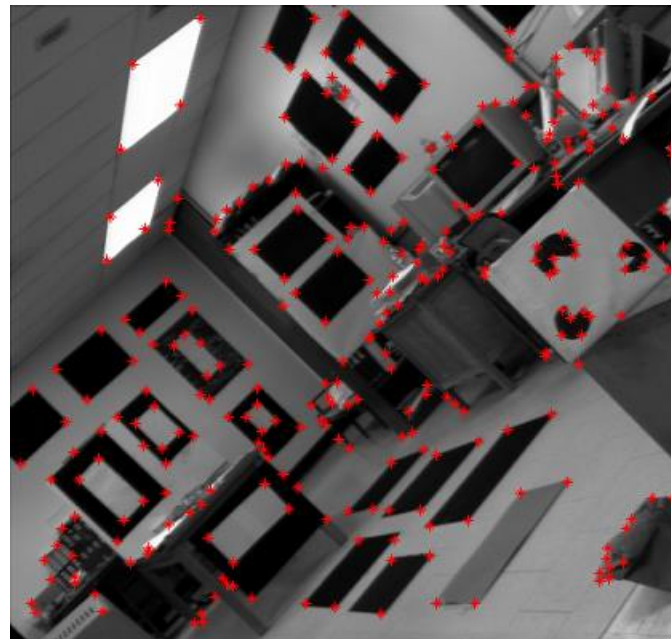
Extraction de contours

Principales méthodes

- **Détection par gradient**
 - Sobel, Prewitt, Roberts (calcul des dérivées).
- **Détection par Laplacien**
 - Cherche les zones où l'intensité change fortement.
- **Canny (la plus utilisée)**
 - Étapes : lissage (Gaussien), calcul du gradient, suppression des non-maxima, seuillage avec hystérésis.
 - Très robuste et précis.

Extraction de coins

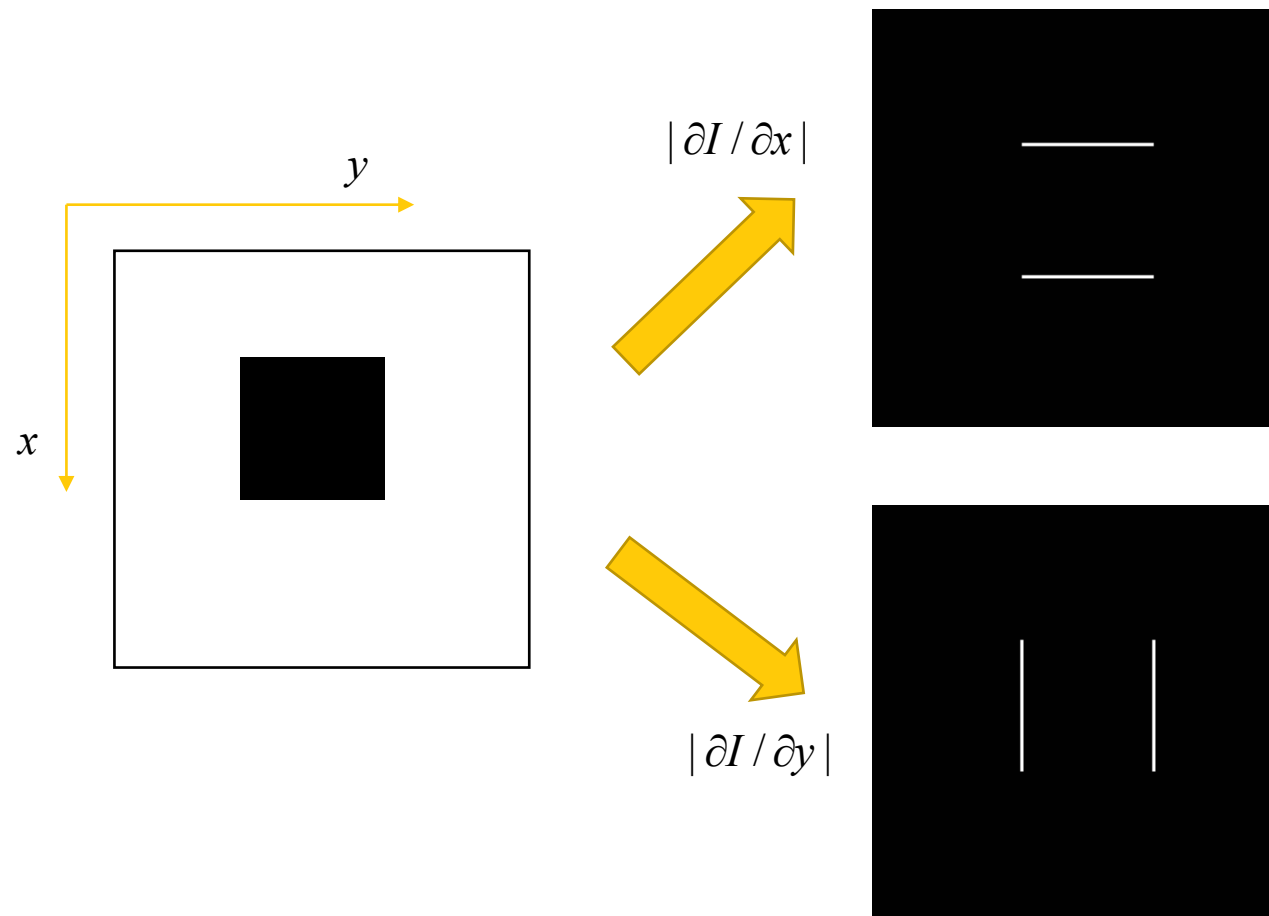
- Un **coin** peut être défini comme **l'intersection de deux arêtes**. Il peut également être défini comme un point pour lequel il existe deux **directions de bord dominantes** dans un voisinage local d'un point.



Les points rouges sont des coins détectés.

Extraction de coins

- Une dérivée en x permet de localiser des contours horizontaux. Une dérivée en y permet de localiser des contours verticaux.



Extraction de coins

Un coin est une zone de l'image qui réagit fortement à la dérivée en x **et** à la dérivée en y .

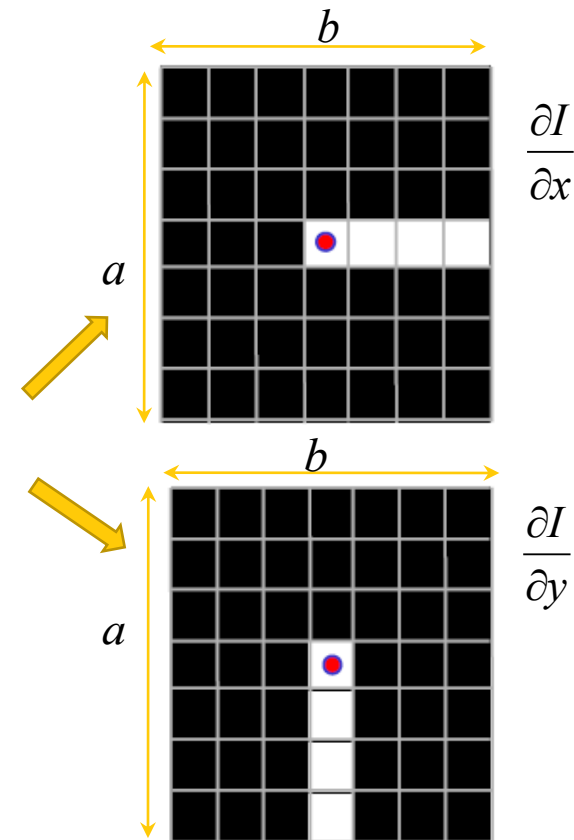
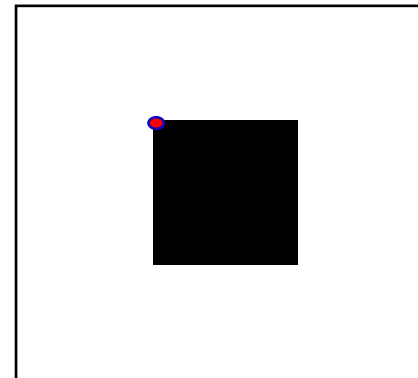
Pour évaluer dans quelle mesure un pixel (x, y) réagit aux deux dérivées, on calcule la **matrice Hessian** :

$$H = \begin{pmatrix} A & B \\ B & C \end{pmatrix};$$

$$A = \sum_{i=x-a/2}^{x+a/2} \sum_{j=y-b/2}^{y+b/2} \frac{\partial I(i, j)}{\partial x} \frac{\partial I(i, j)}{\partial x}$$

$$B = \sum_{i=x-a/2}^{x+a/2} \sum_{j=y-b/2}^{y+b/2} \frac{\partial I(i, j)}{\partial x} \frac{\partial I(i, j)}{\partial y}$$

$$C = \sum_{i=x-a/2}^{x+a/2} \sum_{j=y-b/2}^{y+b/2} \frac{\partial I(i, j)}{\partial y} \frac{\partial I(i, j)}{\partial y}$$



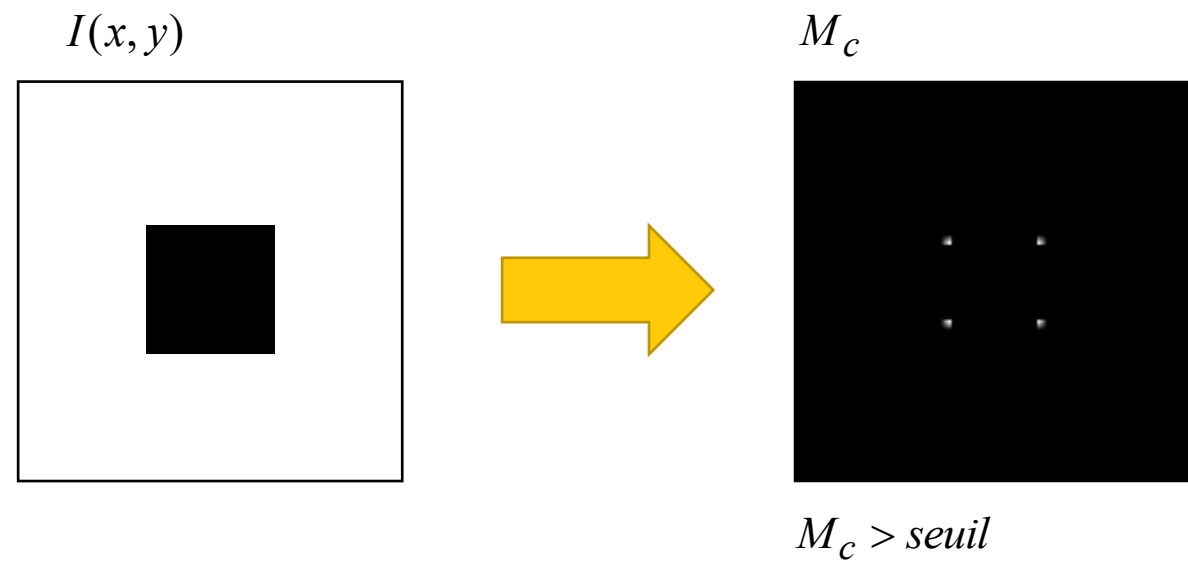
Extraction de coins

- Soit λ_1 et λ_2 les deux valeurs propres de H telles que $\lambda_1 > \lambda_2$. On peut conclure que:
 - Si $\lambda_1 \approx 0$ et $\lambda_2 \approx 0$: le point (x, y) est sur une **région uniforme**.
 - Si $\lambda_1 \gg 0$ et $\lambda_2 \approx 0$: le point (x, y) est sur un **contour**.
 - Si $\lambda_1 \gg 0$ et $\lambda_2 \gg 0$: le point (x, y) est sur un **coin**.
- Pour simplifier les calculs, Harris suggère de calculer le terme suivant pour chaque pixel:

$$M_c = \lambda_1 \lambda_2 - \epsilon (\lambda_1 + \lambda_2)^2 = |H| - \epsilon (\text{tr}(H))^2$$

- ϵ : prend généralement une valeur entre 0.01 et 0.1

Extraction de coins



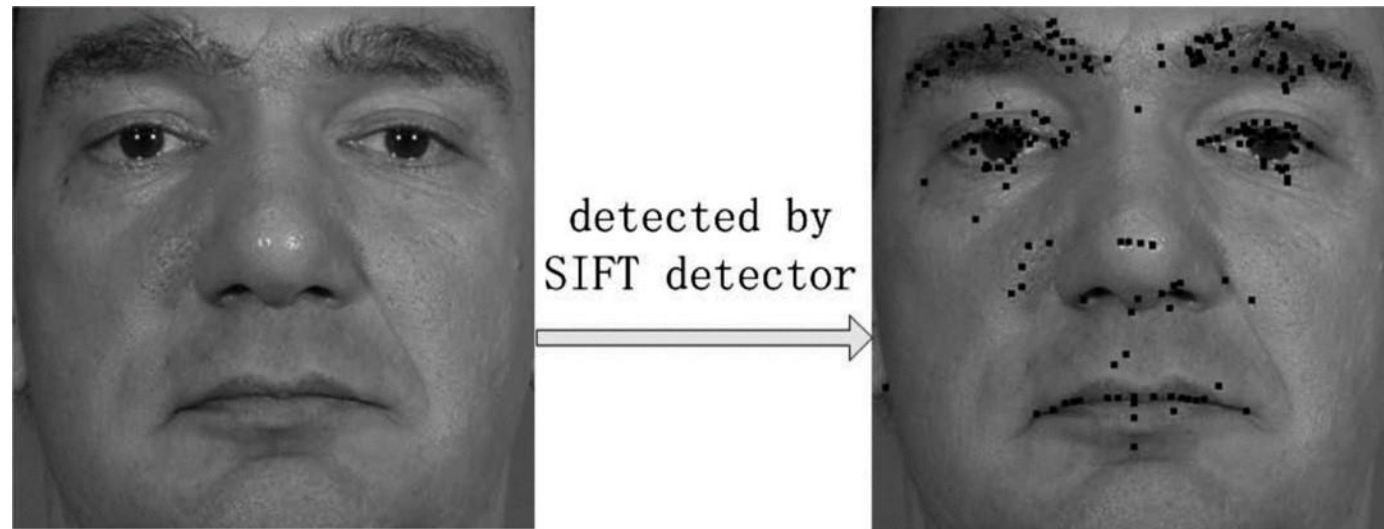
Extraction de points d'intérêt

- Un point d'intérêt (PI) dans une image est un point dont la **position** est bien définie et peut être détecté de manière **robuste**.
- Un PI est généralement associé à un **changement significatif** d'une ou plusieurs propriétés de l'image (ex. l'intensité, la couleur, la texture, etc.).



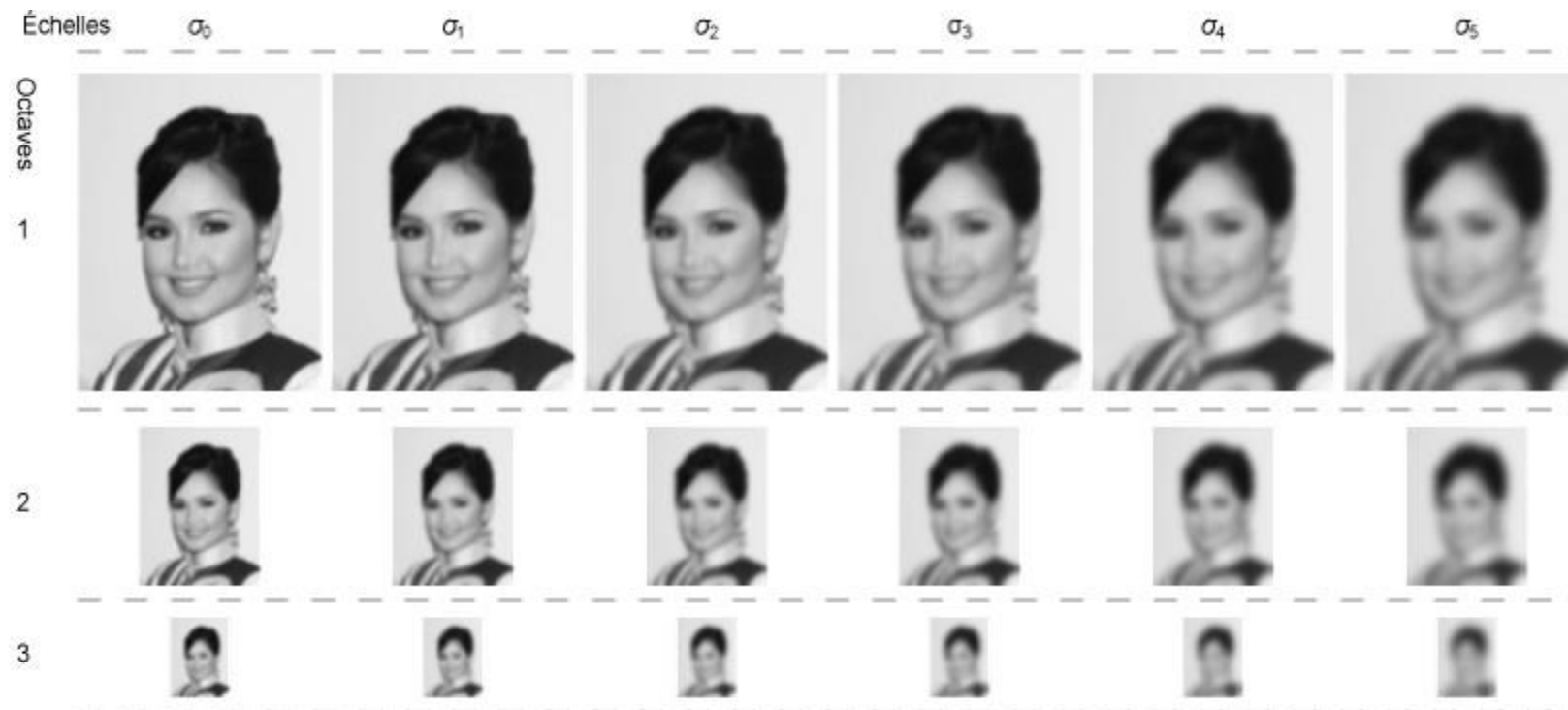
Scale-invariant feature transform (SIFT)

- Un point d'intérêt SIFT (x, y, σ) est caractérisé par sa position (x, y) et son échelle caractéristique σ (D. Lowe 2004).



Scale-invariant feature transform (SIFT)

- Soit I une image et soit un filtre Gaussien G_σ d'échelle σ .
- Filtrant I avec des **filtres Gaussiens avec différents σ** .

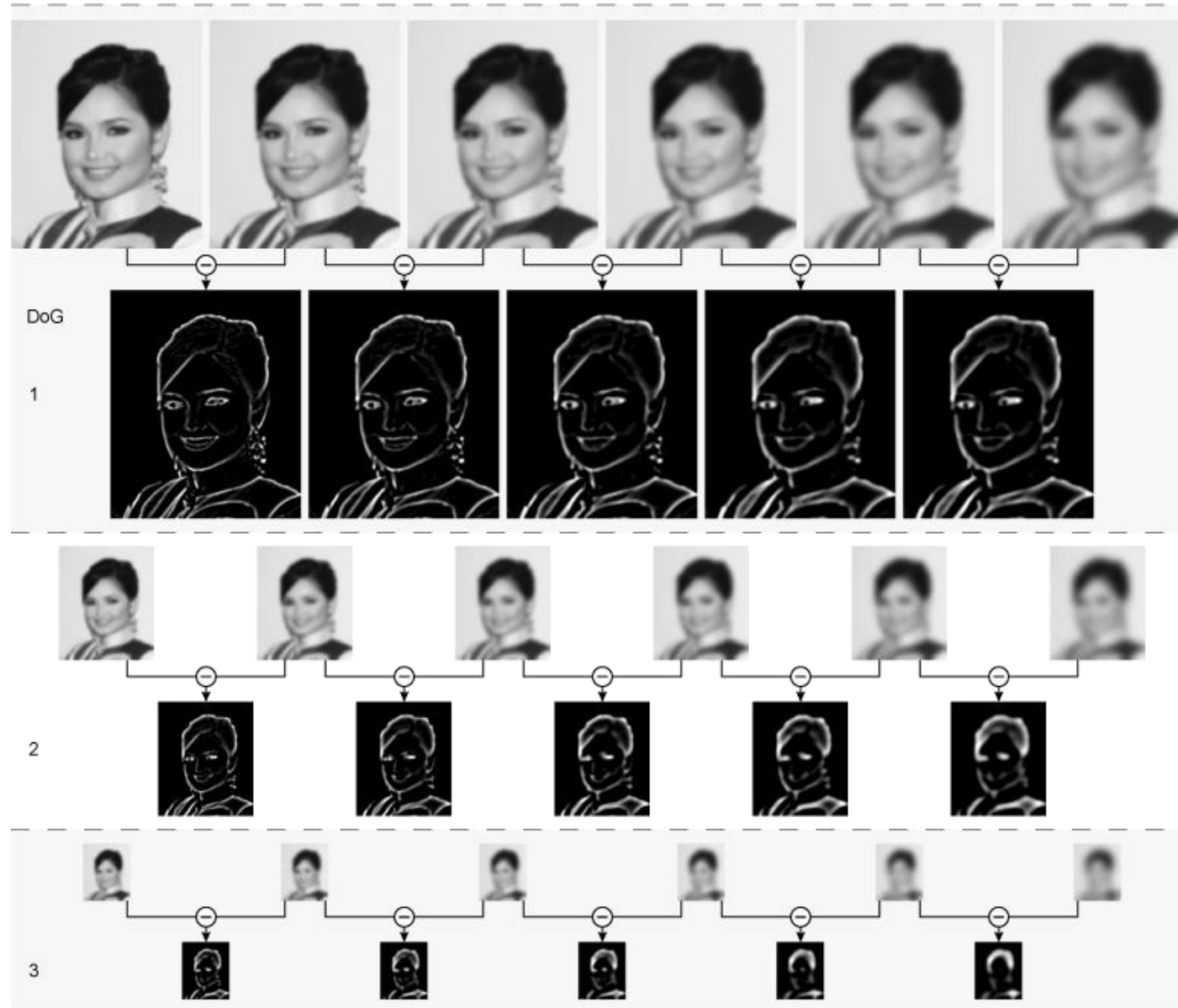


Scale-invariant feature transform (SIFT)

- La détection des points d'intérêt se fait en étudiant l'image appelée **différences de gaussiennes (DoG)**:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma);$$

- Dans l'image D ne persistent plus que **les objets observables** dans des facteurs d'échelle qui varient entre σ et $k\sigma$. k est un paramètre fixe.
- Un **point d'intérêt** candidat est défini comme un point où un **extremum** du DoG est atteint par rapport à ses voisins immédiats.



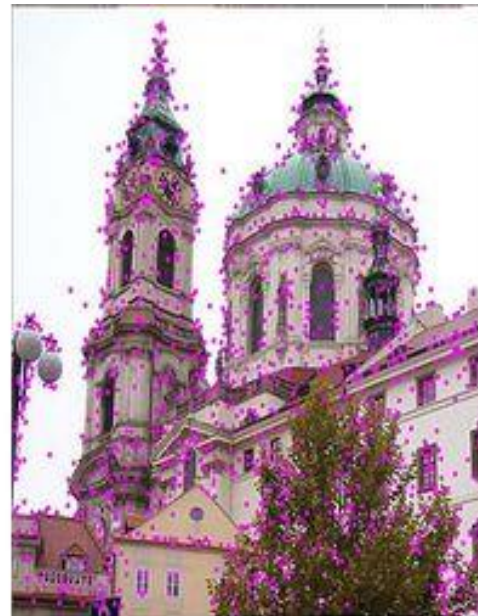
Scale-invariant feature transform (SIFT)

À partir des points détectés (a), on élimine **les points de faible contraste** (b) et les points sur **les arêtes** (contours) (c).

(a)



(b)



(c)



Extraction de régions

- L'extraction de régions dans l'image est communément appelée **segmentation d'images**.



- La plupart des techniques de **segmentation d'images** se basent sur le groupement de couleur/texture (K-moyennes, mélanges de Gaussiennes, coupures de graphes, etc.)

Apprentissage des données visuelles



Quelle est l'identité de cet objet?

Reconnaissance visuelle

- Parmi les tâches les plus importantes qu'on souhaite un **système de vision puisse apprendre**:
 - Analyser une scène.
 - Reconnaître les **objets** qui s'y trouvent.
- Les technologies actuelles en vision ne permettent pas de **nommer** avec exactitude tous les **objets** d'une scène.
- Pourquoi la **reconnaissance visuelle est difficile**?
 - **Grand nombre d'objets** existant dans la nature.
 - **Grande variabilité** à l'intérieur d'une même classe (ex. races de chiens, etc.).

Reconnaissance visuelle

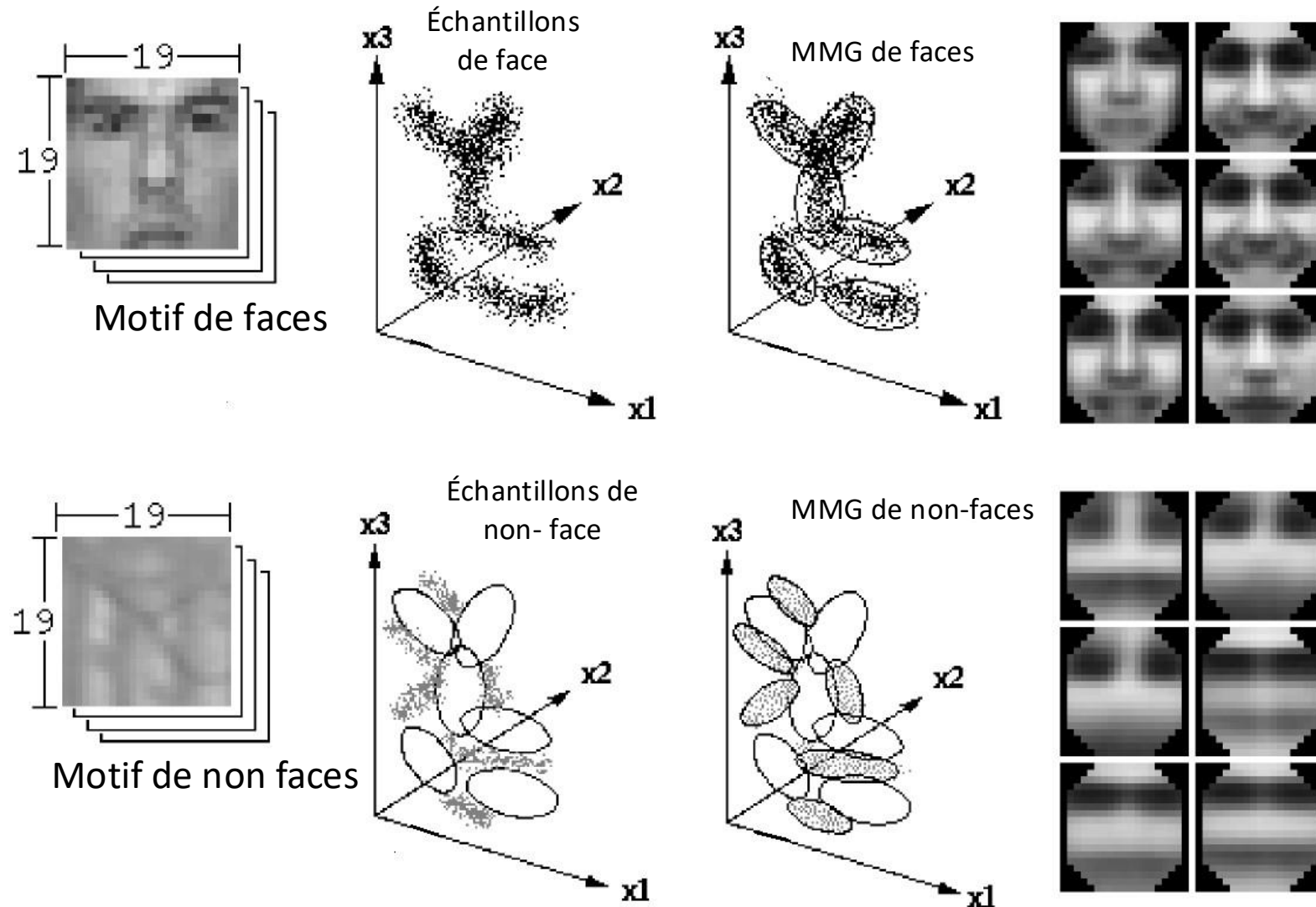
Le problème de reconnaissance visuelle peut être divisé en plusieurs sous-problèmes:

- ☞ Si nous connaissons l'objet recherché dans une image, on parlera de **détection d'objets** (ex. détection de face, piétons).
- ☞ Si nous voulons identifier l'identité d'un objet existant sur une image, on parlera de **reconnaissance d'instance**.
- ☞ On peut faire aussi la **reconnaissance de scène** (ex. nature).

Détection de faces

- La détection de faces est basée sur l'apprentissage supervisé, en utilisant:
 - ➡ Modèles de mélange Gaussien (MMG).
 - ➡ SVM,
 - ➡ Arbres, etc.
- Dans les MMG, on peut utiliser des mélanges de K Gaussiennes pour modéliser la distribution des niveaux de gris des faces et non-faces en d dimensions (la taille d'une image de face).
- Dans l'exemple suivant, $K = 6$ et $d = 19 \times 19$. La distribution des faces et non-faces est modélisée par deux MMGs.

Détection de faces



Détection de faces

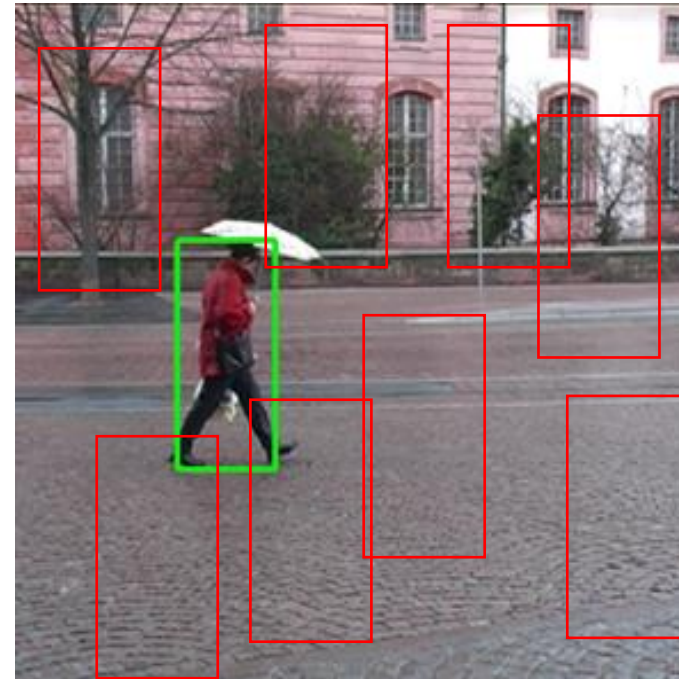
Exemple de détection de faces



Détection d'objets

- Consiste à scanner l'image pour **chercher un objet spécifique (ex. piéton)**.
- On doit décider pour **chaque position** de l'image **si l'objet est présent ou non**.

Exemple: détection de piétons.



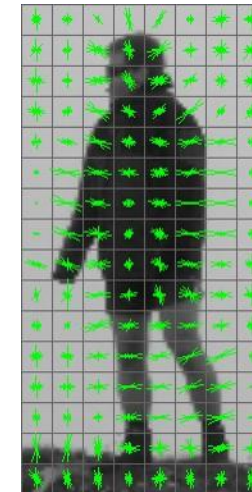
Détection de piétons

- Le descripteur ayant le plus de succès pour la détection de piétons est l'histogramme des orientations du gradient (HOG).
- On utilise l'apprentissage supervisé pour construire un classificateur basé sur les attributs de HOG.

Exemples positifs

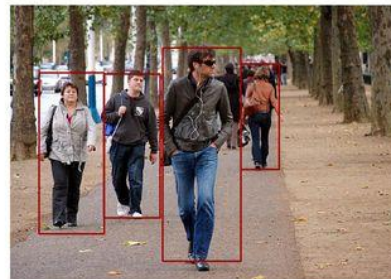
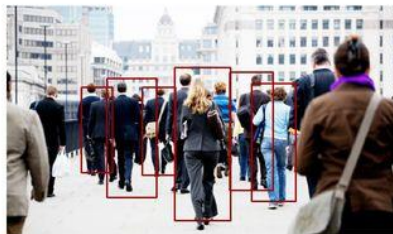
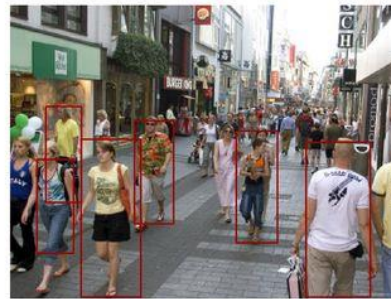


Exemples négatifs



Détection de piétons

On peut utiliser le **SVM** ou la **régression logistique (RL)** pour apprendre le classificateur du piétons.

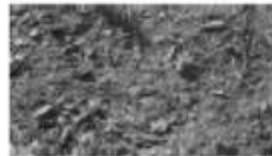


Classification d'images par objets

- Ayant des exemples positifs:



- Ayant des exemples négatifs:



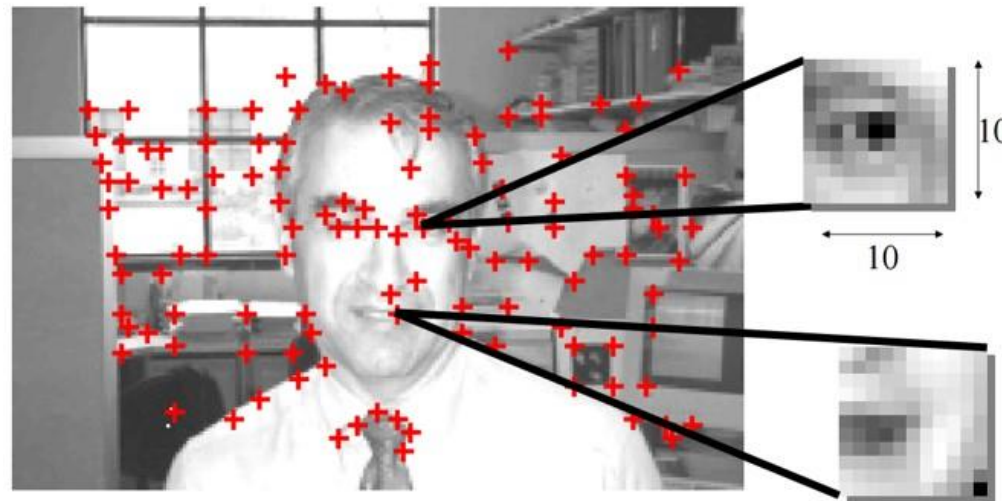
- À quelle catégorie appartient cette image?



?

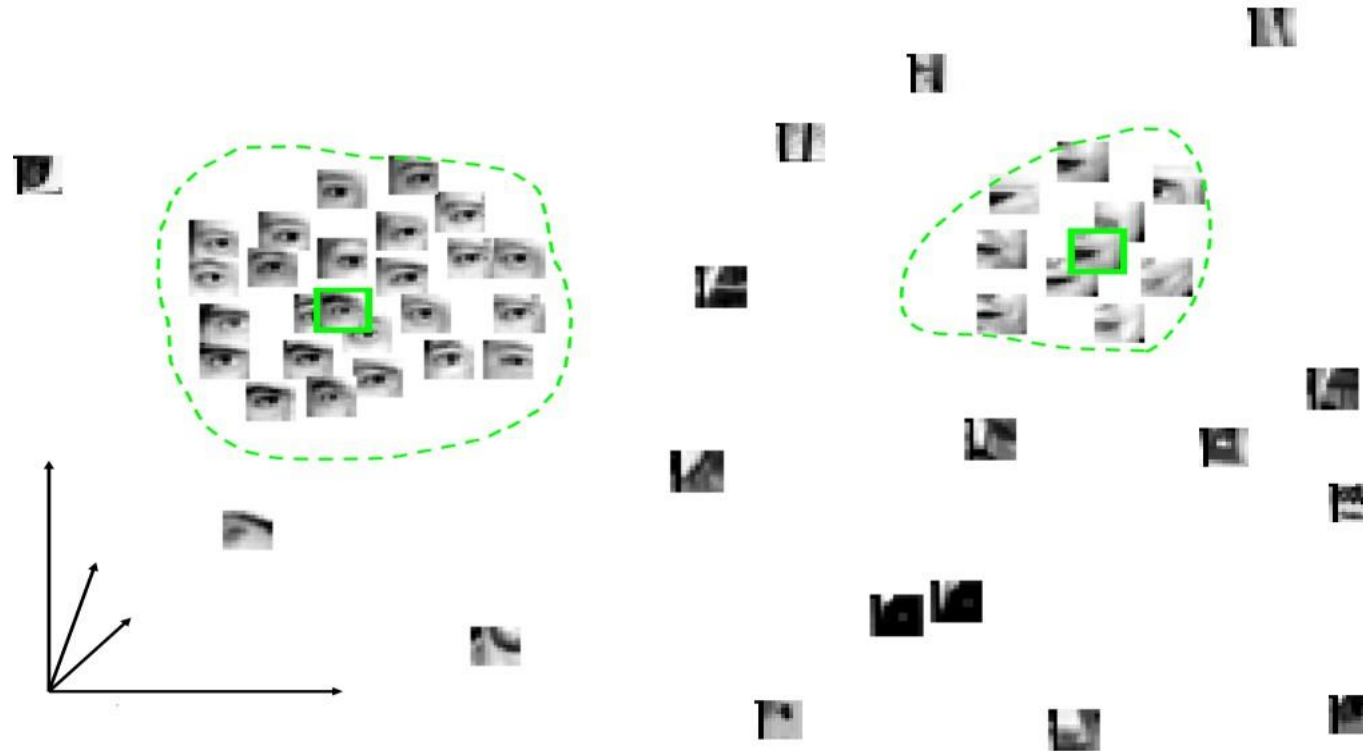
Classification d'images par objets

- On peut utiliser l'approche **sac-à-mots**.
- De manière similaire au texte, on peut définir un vocabulaire visuel V constitué **de mots visuels**.
- **Un mot visuel** peut être défini par exemple **par un descripteur** (ex. SIFT, HOG, etc.).



Approche par sac-à-mots

- On utilise le groupement K -moyennes sur les descripteurs SIFT pour créer **les mots** (centres des groupes).



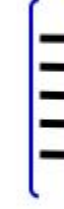
Approche par sac-à-mots

- On utilise **l'apprentissage supervisé** pour apprendre un classificateur (ex. SVM, RL, arbres, etc.) .

Exemples positifs



Exemples négatifs



Classification d'images



bottles



airplanes side



faces



guitars



camel



houses



leaves



cars brad



Motorbikes side



leopards

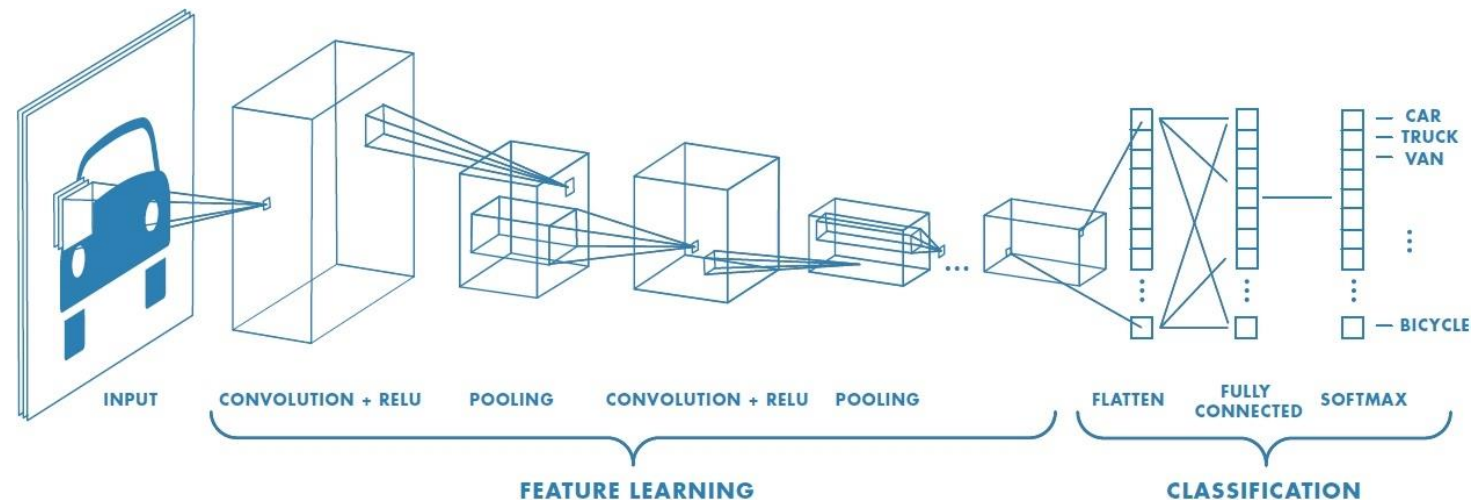
Apprentissage profond pour la reconnaissance visuelle

Apprentissage profond

- Les réseaux de neurones ont été peu utilisés pour classer les données visuelles avant le début des années 2000.
- Après 2010, un grand engouement a resurgi pour **les réseaux de neurones profonds** après les succès obtenus sur la reconnaissance d'objets dans les images.
- Les deux grands facteurs clés de ce succès sont:
 - ☞ La disponibilité **de données annotées** (ex. imageNet, Pascal, avec des millions d'images).
 - ☞ L'augmentation **des capacités de calcul** (ex. GPU, cloud)

Réseaux de neurones convolutionnels

- Un type particulier des réseaux de neurones profond est le **réseau de neurone convolutionnel**, dont le principe de fonctionnement est illustré par le schéma suivant:

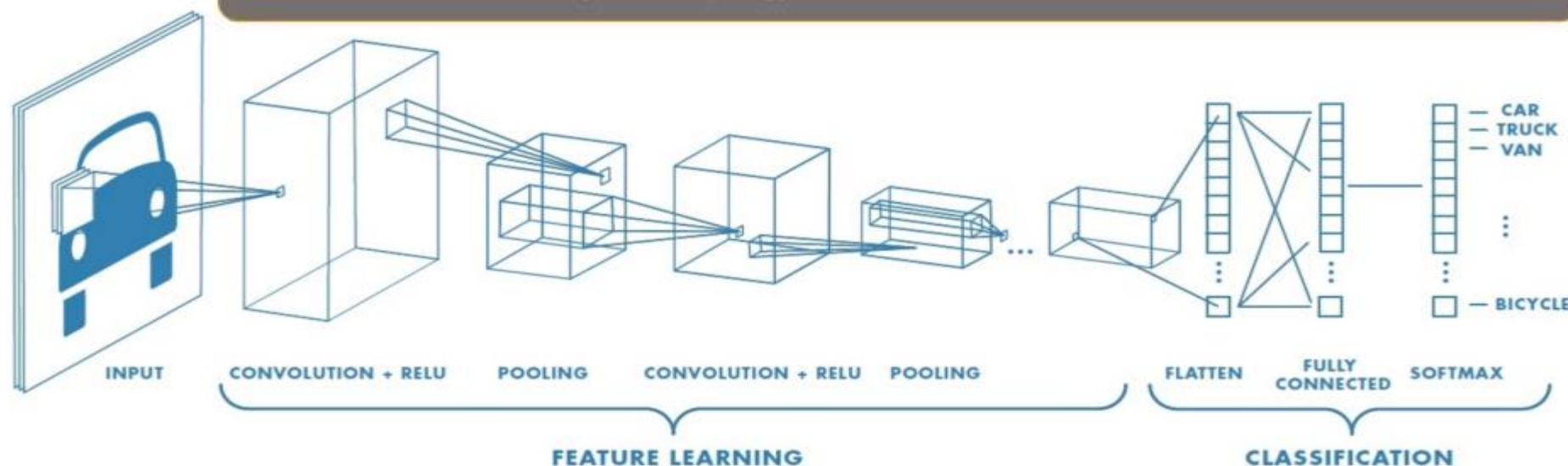


Les couches d'un réseau de neurones convolutionnel (CNN):

- Un réseau de neurones convolutionnel (CNN) possède plusieurs couches cachées qui aident à extraire des informations d'une image.
- Les quatre couches importantes dans un CNN sont :
 - La couche convolutionnelle
 - La couche ReLU
 - La couche de pooling
 - La couche entièrement connectée

Disposition des composants

CNN is a feed forward neural network that is generally used to analyze visual images by processing data with grid like topology. A CNN is also known as a "ConvNet"



The basic layers of CNN.

CNN = Apprendre des motifs (Convolution + Pooling) → Transformer en vecteur (Flatten) → Classifier (Fully Connected + Softmax).

Le CNN est une combinaison de deux blocs de base :

1. **Le Bloc Convolutionnel** : Comprend la couche convolutionnelle, la couche ReLU et la couche de pooling. Ce bloc constitue le composant essentiel pour l'extraction des caractéristiques (Feature-Extraction).
2. **Le Bloc Entièrement Connecté** : Comprend une architecture de réseau de neurones entièrement connectée. Ce bloc effectue la tâche de classification en se basant sur les entrées provenant du bloc convolutionnel.

Comment on utilise des CNN avec python

Construction du CNN

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
```

```
model = models.Sequential([
    layers.Conv2D(8, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(1, activation='sigmoid')
])
```

Compilation du modèle

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Entraînement et validation

```
model.fit(x_train, y_train, epochs=5, batch_size=32, validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f'Précision sur le test : {test_acc:.3f}')
```

Quelques applications

Recherche/indexation d'images

- Malgré que **la reconnaissance visuelle** est dans ces premiers pas, elle a déjà beaucoup d'impact sur **la recherche d'images**.
- On peut faire des requêtes d'images sur le Web pour avoir des images similaires (ex. Google).

Requête



Résultats sur Google



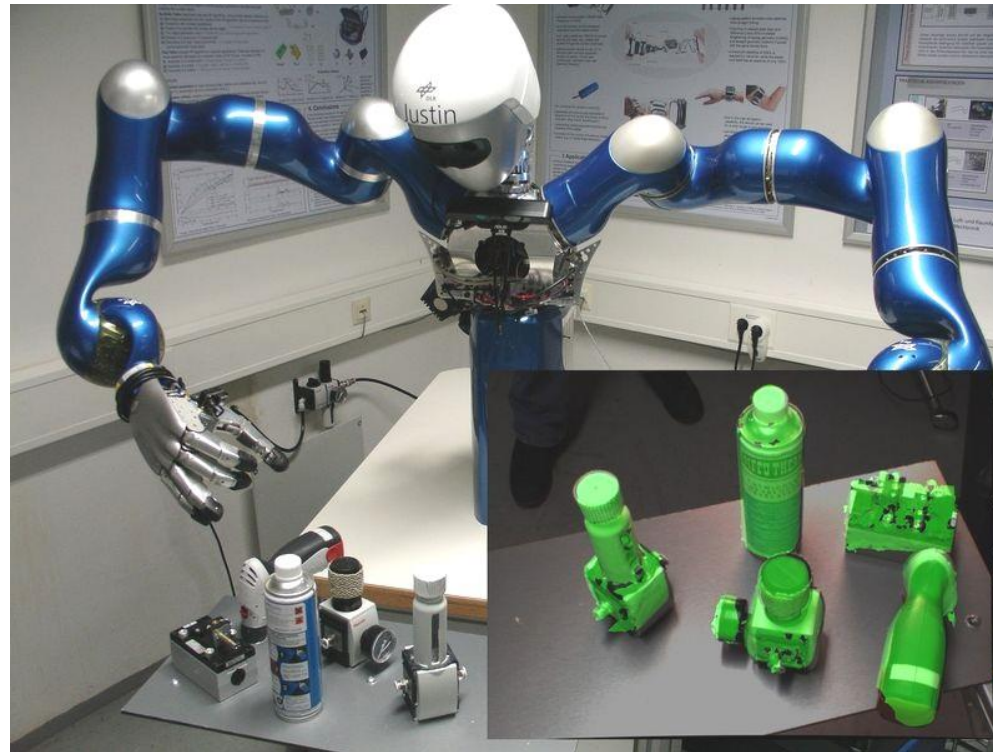
Inspection visuelle

- Détecter des défauts dans des structures:



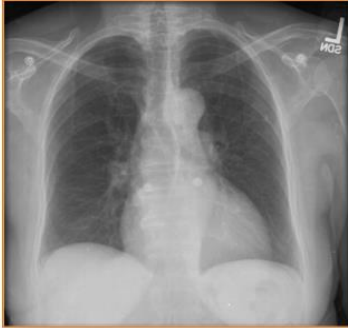
Robots autonomes

- Reconnaissance d'objets pour la navigation automatique:



Diagnostic médical

- Résultats d'annotation avec l'apprentissage profond:

input image	generated annotation	true annotation
	aorta_thoracic / tortuous / mild aorta_thoracic / tortuous	airspace_disease / lung / hilum / right / lung / hilum nodule / lung / hilum / right
	opacity / lung / middle_lobe / right / aorta_thoracic / tortuous opacity / lung / base / left	thoracic_vertebrae_degenerative / mild aorta_tortuous / thoracic_vertebrae_degenerative / mild
	calcified_granuloma / lung / middle_lobe / right / multiple calcified_granuloma / lung / hilum / right	normal normal
	opacity / lung / middle_lobe / right / blood_vessels calcified_granuloma / lung / middle_lobe / right	normal normal

Reconnaissance d'activités en vidéo

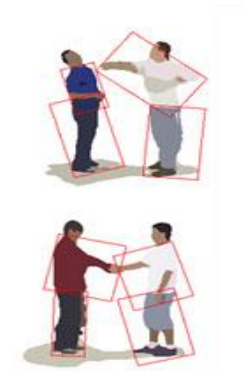
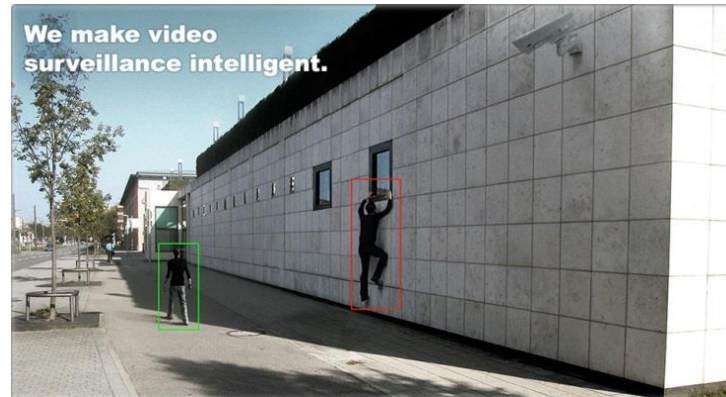
- Les caméras de surveillance sont de plus en plus déployées pour renforcer la sécurité des individus et des biens.



- Les opérateurs humains sont sensibles à la fatigue, distraction, et le nombre de vidéos à analyser ne cesse de grandir:
 - Coût du personnel en croissance!

Reconnaissance d'activités en vidéo

- On peut extraire des **descripteurs de mouvement ou de forme** pour créer des **attributs** de classification d'actions.
- Utiliser l'apprentissage supervisé (ex. SVM, RL, etc.) pour catégoriser les actions.



Laboratoire formatif

Laboratoire formatif

- Charger un dataset réel d'images (dataset Olivetti Faces).
(from sklearn.datasets import fetch_olivetti_faces)
- Extraire des caractéristiques (features) à partir des images :
 - Histogramme de niveaux de gris
 - Extraction de contours (Sobel)
 - Extraction de contours (Canny)
 - Fusion de caractéristiques (Histogramme + Contours)
- Entraîner un modèle de classification supervisée basé sur la Régression Logistique.
- Comparer les performances obtenues selon le descripteur utilisé en fonction de l'ACC
- Afficher au moins 10 images avec leur label.